

```
<script  
src="https://cdn.jsdelivr.net/npm/mermaid/dist/mermaid.min.js">  
</script> <script> mermaid.initialize({ startOnLoad: true }); </script>
```

MySQLでjoinを使うな

2024/10/16 日高幸祐

背景

admin-v2を開発していく中で古野さんから色々とアドバイスをもらう事が多いが、その中で「MySQLはjoinが得意なDatabaseではないから極力joinを使わないようにしてほしい」という話をもらった。

聞いたことない話だったので、気になり、詳しく調べてみることにした

MySQLがjoinに向かない理由

結論

SQLの結合アルゴリズムのNested Loopしか対応していないから

結合アルゴリズムの種類

- Nested Loop
- Hash Join
- Sort Merge Join

Nested Loop

```
<div style="display: flex; justify-content: space-around;"> <div
style="margin-top: 20px; font-size: 80%;"> <ol> <li>結合元のテーブル
の列を取得</li> <li>結合先のテーブルの列を1.で取得したカ
ラムを用いて検索</li> <li>2.で取得した列を結合先のテーブルに
結合</li> </ol> <p style="margin-top: 40px;">つまり計算量はO(N*M)
</p> </div> <div class="mermaid" style="width: 60%;">
sequenceDiagram
    participant A as Table A
    participant B as Table B
    loop
    外側ループ A->>B: 各行を取得
    loop
    内側ループ B->>B: 各行と比較
    alt
    条件が一致
    B->>A: 結合結果に追加
    end
    end
    end
</div> </div> <div
style="font-size: 60%;"> <p> このアルゴリズムは、特に小さなテーブ
ルやインデックスが適切に設定されている場合に有効だが、大規模
なデータセットではパフォーマンスが低下する可能性がある。 </p>
<p> 他の二つのアルゴリズムと比較してこのアルゴリズムのみどのよ
うな比較演算子も使用できる </p> </div>
```

```
<div style="display: flex; justify-content: space-around; font-size: 70%;"> <div> <ol> <li>結合元のテーブルのカラムを取得し、ハッシュテーブルを作成</li> <li>結合先のテーブルのカラムを取得し、ハッシュテーブルを用いて一致する行を検索</li> <li>一致した行を結合結果に追加</li> </ol> <p style="margin-top: 40px;">このアルゴリズムの計算量は $O(N+M)$ で、特に大規模なデータセットに対して効率的です。 </p> </div> <div class="mermaid" style="width: 115%; margin-top: 30px"> sequenceDiagram participant A as Table A participant B as Table B participant H as Hash Table A->>H: ハッシュテーブルを作成 loop 結合先のテーブルを走査 B->>H: 各行をハッシュテーブルで検索 alt 条件が一致 H->>B: 結合結果に追加 end end </div> </div> <div style="font-size: 60%;"> <p> Hash Joinは、特に大規模なデータセットやインデックスがない場合に有効で、Nested Loopよりも効率的に動作します。 </p> <p> ただ、ハッシュ値の比較のため、比較演算子は"="のみ対応しています。また、Hash JoinはHashテーブルを作成して比較を行うため、メモリを多く使用するため、メモリが不足しな
```

Sort Merge Join

<div style="display: flex; justify-content: space-around; font-size: 80%;"> <div> 結合元と結合先のテーブルをそれぞれ結合キーでソート ソートされた両テーブルを順に走査し、結合キーが一致する行を結合 一致した行を結合結果に追加 <p style="margin-top: 40px;"> このアルゴリズムの計算量は $O(N \log N + M \log M)$ で、ソートが必要ですが、結合キーがすでにソートされている場合に非常に効率的です。 </p> </div> <div class="mermaid" style="width: 110%;"> sequenceDiagram participant A as Sorted Table A participant B as Sorted Table B A->>B: ソートされたテーブルを走査 loop 両テーブルを順に比較 alt 結合キーが一致 B->>A: 結合結果に追加 else 結合キーが一致しない A->>B: 次の行へ end end </div> </div> <div style="font-size: 80%;"> <p> Sort Merge Joinは、特に結合キーがすでにソートされている場合や、範囲条件を含む結合に対して有効です。Nested LoopやHash Joinに比べて、ソートが必要のため初期コストが高いですが、ソート済みデータに対しては非常に効率的に動作します。 </p> </div>

各アルゴリズムのデータ量とコストの比較

<div style="font-size: 80%;"> 基本的に"="結合の場合はhash joinが最も早く、容量の大きい場合はmerge join + sortが最も早い。

 容量が多くなるにつれてnested loopの性能は悪くなっていくため、容量の少ないデータ同士の結合、'='演算子以外を用いて結合したい場合を除いて、あまり使いたくない </div> <div style=" text-align: center;"> </div> <div style="font-size: 60%;"> ※ 「hash join」 , 「merge join + sort」 の段差はメモリに乗り切らなくなってディスク側にデータが漏れ出した時のもの。 </div>

主なDatabaseと対応している結合アルゴリズム比較

Database	Nested Loop	Hash Join	Sort Merge Join
MySQL	○	○ (8.0.18以降)	×
PostgreSQL	○	○	○
Oracle	○	○	○

admin-v2でのMySQLで大規模テーブルのjoinを行う時の対処方法

前提条件

- Databaseはいじりたくない
 - 他の様々なプロダクトから参照されているため
 - 大規模なDatabaseのリプレイスプロジェクトが計画されているため

対応方法

- アプリケーション側に負荷をかける
 - 単テーブルずつレコードを抽出し、マージすることでjoinを実現する

補足

- クラウドを用いたシステムの場合スケールイン/アウトのしやすさで考えると、DBよりもAppの方がしやすいので、Appに負荷をかけた方がインフラ的にも楽

まとめ

- RDBのテーブル結合アルゴリズムには3種類存在する
 - Nested Loop
 - Hash Join
 - Sort Merge Join
- MySQLはNested Loopしか対応していないため、大量のデータを扱う場合はあまり向かないかもしれない
 - ただ、8.0.18以降はHash joinが使えるようになっているので、結合アルゴリズム上はMySQLだけ使い勝手が悪いという状況はなさそう

```
SET optimizer_switch = 'hash_join=on';
```

参考

- [Nested Loop/Hash/Sort Merge結合の違いとパフォーマンス比較](#)
- [Queries in PostgreSQL: 7. Sort and merge](#)
- [\[SQL\]JOINの3種類のアルゴリズムについて](#)